第39课 动态规划经典题（数字三角形）【选学】

> **本课为选学内容**，旨在为对动态规划感兴趣的同学提供进一步探索的机会。学不懂没关系，能跟着思考就是最大的收获！已经走到第39课的你，本身就非常了不起了！

一、课程信息

项目	内容
课次	第 39 课 / 共 40 课
主题	动态规划经典题——数字三角形【选学】
前置知识	第38课 DP入门（斐波那契数列、爬楼梯问题）、二维数组
授课时长	100 分钟

教学目标

1. **能用二维数组存储数字三角形**，并理解三角形的行列对应关系。
2. **能写出数字三角形的状态转移方程**，并用自底向上的方式编写完整代码，正确求出最大路径和。
3. **能用"找状态找转移定初值写代码"四步法**，独立分析一道简单DP题的解题思路。

二、教学重点与难点

教学重点

- 数字三角形问题的建模：如何把三角形存进二维数组。
- 状态转移方程的推导：`dp[i][j] = max(dp[i+1][j], dp[i+1][j+1]) + a[i][j]`（自底向上）。
- DP四步法的巩固运用。

教学难点

- 理解"自底向上"和"自顶向下"两种思路的区别（重点教自底向上，更直观）。
- 二维数组下标的对应关系，不越界。
- 从具体例子抽象出一般的转移方程。

三、课堂时间分配（共 100 分钟）

时间	环节	内容	备注
0-5 min	暖场回顾	回顾第38课DP入门，复习DP四步法	互动提问
5-15 min	新课引入	"下山捡金币"故事引入数字三角形	讲解 ≤ 10 min
15-25 min	知识讲解	用二维数组存三角形 + 手动模拟求解	黑板画图
25-40 min	例题精讲	例题1：最简单的3行三角形（手算+代码）	讲解 ≤ 15 min
40-55 min	动手实践	练习题1-2，学生上机编写	巡堂指导
55-70 min	例题进阶	例题2：标准数字三角形（n行通用代码）	讲解 ≤ 15 min
70-85 min	动手实践	练习题3（选做练习题4）	巡堂指导
85-95 min	拓展了解	01背包思想简介（仅讲故事，不要求掌握）	鼓励为主
95-100 min	总结预告	总结DP学习心得 + 预告下节课总复习	颁奖预告

> 讲解段最长不超过 25 分钟，穿插提问和手动模拟，保持互动。

四、讲稿提纲

1. 暖场回顾（5 分钟）

> 同学们好！上节课我们学了一个超厉害的思想——动态规划，还记得吗？
>
> 我来考考大家：DP四步法是哪四步？
> 1. **找状态**——我要记录什么？
> 2. **找转移**——当前状态怎么从之前的状态算出来？
> 3. **定初值**——最开始的值是多少？
> 4. **写代码**——把上面的想法变成程序！
>
> 上节课我们用DP解决了斐波那契和爬楼梯，今天我们来挑战一道更有意思的经典题——**数字三角形**！
>
> **今天是选学内容哦**，就算暂时不理解也完全没关系，能跟着一起思考，你就已经超越了大多数同龄人啦！

2. 故事引入——下山捡金币（10 分钟）

> 想象你站在一座金币山的山顶。山上铺满了金币，排成一个三角形的样子：
>
> `2 3 4 6 5 7 4 1 8 3`
>
> 你从山顶出发，每次只能往**左下**或**右下**走一步，一直走到山脚。经过的每个位置都能捡起金币。
>
> 问题来了：**怎么走，才能捡到最多的金币呢？**
>
> 这就是大名鼎鼎的"数字三角形"问题！

先想笨办法

> 最笨的方法是什么？——把所有可能的路线都试一遍，比较哪条路金币最多。
>
> 但如果山有20层，路线就有 2^19 = 524288 条！太多了，算到天黑都算不完。
>
> 还记得上节课的经验吗？遇到这种"重复计算"的问题，就该请出我们的好朋友——**动态规划**！

换一个聪明的想法：从山脚开始想

> 我们换个角度：不从山顶出发，而是**从山脚开始，往上想**。
>
> 站在倒数第二行的每个位置，你只需要看脚下的两条路——左下和右下——哪条金币更多，就选哪条。
>
> 就像你在商场选电梯一样：左边的电梯排了3个人，右边排了8个人，你肯定选左边（人少的）。我们这里相反，要**选金币多的**！
>
> 这样一层一层往上推，最后到山顶，就能知道最多能捡多少金币了！

3. 知识讲解——用二维数组存三角形（10 分钟）

> 我们先来解决第一个问题：怎么把三角形存到电脑里？
>
> 用**二维数组** `a[i][j]`：
>
> - `i` 表示第几行（从第1行开始）
> - `j` 表示这一行的第几个数
> - 第 `i` 行恰好有 `i` 个数
>
> `行号 三角形 数组存储 1 2 a[1][1]=2 2 3 4 a[2][1]=3 a[2][2]=4 3 6 5 7 a[3][1]=6 a[3][2]=5 a[3][3]=7 4 4 1 8 3 a[4][1]=4 a[4][2]=1 a[4][3]=8 a[4][4]=3`

自底向上的DP思路

> 再来一个数组 `dp[i][j]`，表示**从第 i 行第 j 个位置出发，走到山脚，最多能捡多少金币**。
>
> **初始值**：最后一行就是山脚，不用再往下走了，所以 `dp[4][j] = a[4][j]`。
>
> **状态转移**（从倒数第二行开始，往上推）：
> `dp[i][j] = max(dp[i+1][j], dp[i+1][j+1]) + a[i][j]`
> 翻译成大白话：**"我这个位置的最大金币 = 下面两条路中金币多的那条 + 我自己位置的金币"**
>
> （在黑板上手动模拟整个过程，让学生跟着一起算）
>
> `原始三角形： dp数组（自底向上填）： 2 20 3 4 11 14 6 5 7 10 13 15 4 1 8 3 4 1 8 3 初始值就是本身`
>
> 所以最多能捡到 **20** 个金币！路线是 2478。

4. 拓展了解——01背包思想（仅故事介绍，不要求掌握）

> 如果今天的内容你已经完全搞懂了，老师再给你讲一个有趣的故事。
>
> 想象你要去野营，背包只能装 10 斤的东西。桌上有这些物品：
> - 帐篷：5斤，价值 8 分
> - 食物：3斤，价值 5 分
> - 水壶：4斤，价值 6 分
> - 玩具：2斤，价值 3 分
>
> 每样东西只能选一次（要么带，要么不带），怎么装才能让总价值最大？
>
> 这就是著名的 **01背包问题**——"0"表示不选，"1"表示选。它也是用DP来解决的！
>
> 这个问题我们今天不写代码，大家只需要知道有这样一类问题就好。以后在信息学竞赛的学习中，你们一定会再次遇到它的！

七、常见错误与应对

错误1：数组下标从0开始导致混乱

```
// 错误：用a[0][0]存山顶，下标容易搞混
a[0][0] = 2;

// 正确：从下标1开始，a[1][1]是山顶，和题目描述一致
a[1][1] = 2;
```

应对：告诉学生这类题统一从下标1开始，数组多开几个，宁可浪费一点空间也不要搞混。

错误2：循环方向搞反

```
// 错误：自底向上应该i从大到小，写成从小到大了
for (int i = 1; i <= n - 1; i++)

// 正确：从倒数第二行（n-1）开始，往上推到第1行
for (int i = n - 1; i >= 1; i--)
```

应对：提醒学生画图——我们是从山脚往山顶推，所以行号从大到小。

错误3：内层循环范围错误

```
// 错误：第i行有i个数，不是n个
for (int j = 1; j <= n; j++)

// 正确：
for (int j = 1; j <= i; j++)
```

应对：让学生回忆三角形的形状——第1行1个数，第2行2个数……第i行i个数。

错误4：转移方程写反方向

```
// 错误（这是自顶向下的方程，和我们的循环方向不匹配）：
dp[i][j] = max(dp[i - 1][j - 1], dp[i - 1][j]) + a[i][j];

// 正确（自底向上）：
dp[i][j] = max(dp[i + 1][j], dp[i + 1][j + 1]) + a[i][j];
```

应对：统一教自底向上，让学生记住口诀——**"我站这里，看脚下两条路，选大的加自己"**。

错误5：忘记初始化最后一行

```
// 错误：直接开始转移，没有给最后一行赋初值
// dp数组全是0，结果当然不对

// 正确：先把最后一行的dp值设好
for (int j = 1; j <= n; j++)
 dp[n][j] = a[n][j];
```

应对：类比"爬楼梯的第1阶和第2阶"——DP必须有起点（初始值），不然推不动。

九、下节课预告

> 同学们！下节课就是我们这个学期的**最后一课**啦！
>
> **第40课：总复习 + 模拟赛 + 颁奖典礼**
>
> 我们会一起回顾这40节课学到的所有知识，从最开始的 `Hello World` 到现在的动态规划——你们的成长真的非常了不起！
>
> 然后会有一场小小的模拟赛，检验一下大家的学习成果。最后是激动人心的**颁奖环节**！
>
> 请大家提前复习之前的笔记和代码，以最好的状态迎接最后一课吧！

附录：板书设计参考

```
┌──────────────────────────────────────────────────────────┐
│ 第39课 数字三角形【选学】 │
│ │
│ 下山捡金币： DP四步法： │
│ 2 1. 找状态：dp[i][j] │
│ 3 4 2. 找转移：max(下左,下右)+自己 │
│ 6 5 7 3. 定初值：最后一行=原值 │
│ 4 1 8 3 4. 写代码：两层循环 │
│ │
│ 自底向上填dp表： 状态转移方程： │
│ 20 dp[i][j] = max(dp[i+1][j], │
│ 11 14 dp[i+1][j+1]) + a[i][j] │
│ 10 13 15 │
│ 4 1 8 3 口诀：看脚下两条路， │
│ 选大的加自己！ │
│ 答案：dp[1][1] = 20 │
└──────────────────────────────────────────────────────────┘
```

> **教师备注：**
>
> 1. 本课为选学内容，全程保持鼓励性语气，不要让学生有"必须学会"的压力。
> 2. 如果大部分学生在例题1之后就感到吃力，可以多花时间巩固例题1和练习题1-2，不必强推例题2。
> 3. 01背包部分仅作为故事讲述，不要求学生理解算法细节，激发兴趣即可。
> 4. 练习题4是给学有余力的同学准备的，大部分同学完成练习题1-3即可。
> 5. 下节课是最后一课（总复习+模拟赛+颁奖），本课结束时要营造期待感，让学生带着愉快的心情迎接最后一课。